Tarefa 13: Avaliação da Escalabilidade com Afinidade de Threads

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

A **afinidade de threads** define em quais CPUs cada thread pode executar. Sem afinidade, o sistema operacional escolhe e pode migrar as threads a qualquer momento; com afinidade, cada thread fica restrita a um *lugar* (*place*), que pode ser uma CPU lógica, um núcleo ou um processador inteiro.

O objetivo desta tarefa é avaliar como as políticas de afinidade do OpenMP, controladas por `OMP_PROC_BIND` e `OMP_PLACES`, alteram a escalabilidade forte do código de Navier-Stokes 2D (versão 4 da tarefa 12) em um nó do NPAD.

2 Metodologia

Ambiente. Nó da partição `amd-512` do NPAD, com 2 processadores AMD EPYC 7713 (128 núcleos físicos no total) e 2 threads por núcleo (SMT), somando 256 CPUs lógicas.

Código. Versão 4 da tarefa 12, sem alteração no cálculo. A única mudança foi permitir escolher a política de afinidade e registrar em que CPU cada thread executa, para contar migrações. Nos experimentos, a política e os lugares foram definidos pelas variáveis de ambiente.

Experimento. Malha 2048×2048, 210 passos de tempo, de 1 a 256 threads (potências de 2) e 5 repetições de cada caso. O *speedup* é calculado em relação à execução sem afinidade com 1 thread (2,667 s). As afinidades avaliadas estão na Tabela 1.

Foram testadas 9 combinações: `false`; `true` com `cores`; `close` e `spread` com `threads`, `cores` e `sockets`; e `master` com `cores`. Os scripts e o código completo estão no Apêndice.

3 Resultados

A Tabela 2 mostra o speedup médio de cada combinação e a Figura 1 mostra as mesmas curvas. Em todas as execuções o erro máximo em relação à solução analítica foi o mesmo ($1{,}1 \times 10^{-7}$), ou seja, a afinidade muda apenas o tempo, não o resultado.

OMP_PROC_BIND	O que faz
false	Sem afinidade: o sistema operacional pode migrar as threads entre CPUs.
true	Cada thread fica fixa no lugar onde foi criada, seguindo a ordem dos lugares.
close	Threads em lugares consecutivos, o mais perto possível da thread mestre (compacto).
spread	Threads distribuídas por igual entre todos os lugares (espalhado pelo nó).
master	Todas as threads no mesmo lugar da thread mestre.
OMP_PLACES	**Lugar**
threads	Uma CPU lógica (uma thread de hardware).
cores	Um núcleo físico, com suas 2 threads SMT.
sockets	Um processador inteiro (64 núcleos); a thread ainda pode se mover dentro dele.

Tabela 1: Políticas de afinidade e tipos de lugar avaliados.

Política | lugares	1	2	4	8	16	32	64	128	256
false	1,0	1,9	3,8	7,5	14,1	27,5	32,5	14,6	15,4
true |cores	1,1	2,0	2,8	2,9	4,3	28,1	43,8	76,8	78,3
close |threads	0,97	0,99	1,9	2,7	2,9	4,3	25,7	39,3	71,0
close |cores	1,0	2,0	2,8	2,9	4,3	28,9	44,6	76,2	77,9
close |sockets	0,99	1,8	3,7	7,0	8,2	23,5	41,6	24,2	28,7
spread |threads	0,97	1,9	3,7	7,4	14,5	28,0	51,1	75,3	77,3
spread |cores	1,0	1,8	3,8	7,5	14,4	28,3	50,5	75,9	76,8
spread |sockets	0,99	1,9	4,1	6,6	7,5	26,4	41,5	26,0	34,3
master |cores	1,0	1,0	0,84	0,55	0,45	0,21	0,10	0,05	0,02

Tabela 2: Speedup médio (5 repetições) por número de threads.

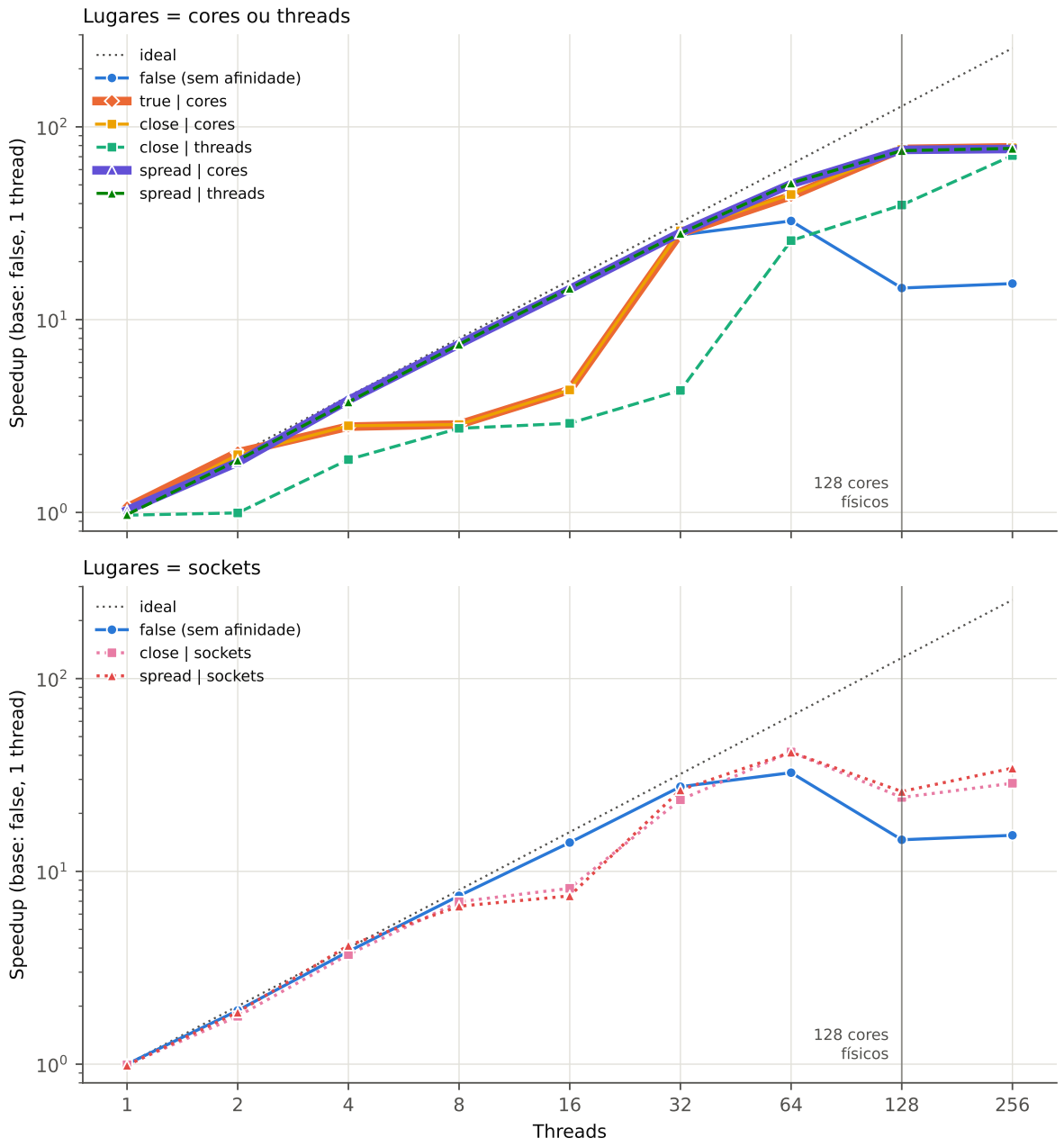


Figura 1: Speedup em função do número de threads. Acima, lugares `cores` e `threads`; abaixo, lugares `sockets`. As curvas de `true|cores` e `close|cores` coincidem, assim como as de `spread|cores` e `spread|threads`.

Com 1 thread todos os casos ficam entre 0,97 e 1,06, o que dá uma ideia do ruído de medição. Nos casos sem fixação por núcleo, os tempos variaram mais entre as repetições.

4 Análise

- **Sem afinidade (`false`).** O sistema operacional escolhe onde cada thread executa. Funciona bem até 32 threads (speedup 27,5), mas com 128 e 256 threads o speedup cai para 14,6 e 15,4 e os tempos ficam instáveis. Sem controle, as threads acabam

mal distribuídas entre os núcleos, e a mais lenta atrasa todas.

- `spread`. Espalha as threads por todo o processador e é a política que melhor se mantém próxima do ideal em toda a faixa: speedup de 14,4 com 16 threads, 28,3 com 32 e 50,5 com 64 (contra 32,5 sem afinidade).
- `true` e `close` com `cores`. Agrupam as threads nos primeiros núcleos. Com poucas threads isso atrapalha: o speedup fica parado em cerca de 2,8 com 4 e 8 threads e chega a só 4,3 com 16. A partir de 32 threads o desempenho se equipara ao dos demais (28,1 a 28,9). As duas políticas deram o mesmo resultado.
- **Todos os núcleos em uso (128 threads).** Qualquer fixação por núcleo dá praticamente o mesmo resultado (75 a 77). Com 256 threads (2 threads por núcleo, SMT) o ganho é de apenas 2% (78,3), então usar o SMT não compensa neste código.
- `close` com `threads`. Pelos resultados, as threads ocupam as duas CPUs lógicas de cada núcleo antes de passar para o próximo, e por isso usam só metade dos núcleos: com 2 threads não há ganho (0,99) e com 16 o speedup é só 2,9. Só se aproxima dos demais com 256 threads (71).
- **Lugares** `sockets`. A thread fica presa ao processador, mas o sistema ainda escolhe o núcleo. O resultado fica entre o `false` e a fixação por núcleo: com 128 e 256 threads o speedup é de 24 a 34, melhor que sem afinidade (14,6 e 15,4), mas bem abaixo da fixação por núcleo (76 a 78).
- `master`. Coloca todas as threads no mesmo núcleo. Com 2 threads não há ganho (1,01) e, com mais threads, o programa fica mais lento que o serial: com 256 threads o cálculo leva 109,6 s, cerca de 41 vezes mais que com 1 thread (Figura 2).

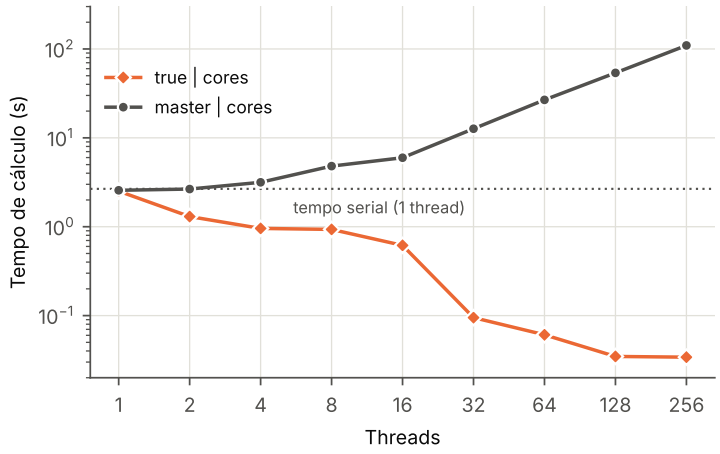


Figura 2: Tempo de cálculo de `master|cores` comparado a `true|cores`. Com `master`, mais threads significam mais tempo.

5 Conclusão

A afinidade não faz diferença com 1 thread, mas importa cada vez mais conforme o número de threads cresce: com 128 threads o speedup é 14,6 sem afinidade e de 75 a 77 fixando cada thread em um núcleo.

- Usar `spread` com lugares `cores` (ou `threads`) quando houver menos threads que núcleos: é a política com melhor desempenho em toda a faixa.
- Com todos os núcleos em uso, qualquer fixação por núcleo serve (`true`, `close` ou `spread` com `cores`). Usar o SMT (256 threads) quase não compensa.
- Evitar `false` com muitas threads (acima de 32), lugares `sockets`, `close` com `threads` e `master`.

Os resultados valem para uma única malha (2048×2048).

A Código-Fonte

```c
/*
 * Navier-Stokes 2D apenas com viscosidade (sem pressao e sem forcas externas)
 * TAREFA 13 (OpenMP): v4 (regiao paralela unica + kernel SIMD) com AFINIDADE DE THREADS
 *
 * Argumentos: <t_final> [N] [bind] (N padrao 1024, bind padrao env)
 *
 * Compilar: gcc -O3 -Wall -fopenmp -o tarefa13 tarefa13.c -lm -lmpascalops
 * Exemplos: OMP_NUM_THREADS=16 OMP_PROC_BIND=close OMP_PLACES=cores ./tarefa13 0.001 2048
 *           OMP_NUM_THREADS=16 OMP_PLACES=cores ./tarefa13 0.001 2048 spread
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <sched.h>
#include <omp.h>
#include "pascalops.h"

#define IDX(i, j, N) ((size_t)(i) * (N) + (j))

#define MODO  2
#define N_GRID 1024
#define NU    0.01

typedef enum { B_ENV, B_MASTER, B_CLOSE, B_SPREAD } Bind;

typedef struct {
    double *u, *v, *un, *vn;
    int    N, nsteps, saida;
    double dt, r, dA;
    long   total;
    double t_passo;
    int    *cpu_ini, *cpu_fim; /* CPU de cada thread no inicio e no fim */
    long   *migr;              /* migracoes observadas por thread */
} Ctx;

static double g_um, g_vm, g_mu, g_mv, g_en;

static inline void calc_linha(const double *restrict u, const double *restrict v,
                              double *restrict un, double *restrict vn,
                              int N, int i, double r)
{
    const int im = (i == 0)  ? N - 1 : i - 1;
    const int ip = (i == N - 1) ? 0  : i + 1;
    const size_t c = (size_t)i * N;
    const size_t a = (size_t)im * N;
    const size_t b = (size_t)ip * N;

    #pragma omp simd
    for (int j = 1; j < N - 1; j++) {
        double lap_u = u[b + j] + u[a + j] + u[c + j + 1] + u[c + j - 1]
                     - 4.0 * u[c + j];
        double lap_v = v[b + j] + v[a + j] + v[c + j + 1] + v[c + j - 1]
```

```
                    - 4.0 * v[c + j];
        un[c + j] = u[c + j] + r * lap_u;
        vn[c + j] = v[c + j] + r * lap_v;
    }

    const int bordas[2] = {0, N - 1};
    for (int k = 0; k < 2; k++) {
        const int j = bordas[k];
        const int jm = (j == 0)  ? N - 1 : j - 1;
        const int jp = (j == N - 1) ? 0  : j + 1;
        double lap_u = u[b + j] + u[a + j] + u[c + jp] + u[c + jm]
                    - 4.0 * u[c + j];
        double lap_v = v[b + j] + v[a + j] + v[c + jp] + v[c + jm]
                    - 4.0 * v[c + j];
        un[c + j] = u[c + j] + r * lap_u;
        vn[c + j] = v[c + j] + r * lap_v;
    }
}

static void integra(Ctx *c)
{
    const int N     = c->N;
    const int nsteps = c->nsteps;
    const long total = c->total;
    const double dt = c->dt, dA = c->dA;

    const double *cu = c->u, *cv = c->v;
    double      *nu_ = c->un, *nv_ = c->vn;
    const int tid = omp_get_thread_num();

    int  ult = sched_getcpu();
    long mig = 0;
    c->cpu_ini[tid] = ult;

    for (int n = 0; n <= nsteps; n++) {

        int cpu = sched_getcpu();
        if (cpu != ult) { mig++; ult = cpu; }

        if (n % c->saida == 0 || n == nsteps) {
            #pragma omp single
            { g_um = 0; g_vm = 0; g_mu = 0; g_mv = 0; g_en = 0; }

            #pragma omp for schedule(static) \
                    reduction(max:g_um, g_vm) reduction(+:g_mu, g_mv, g_en)
            for (long k = 0; k < total; k++) {
                double fu = fabs(cu[k]), fv = fabs(cv[k]);
                if (fu > g_um) g_um = fu;
                if (fv > g_vm) g_vm = fv;
                g_mu += cu[k];
                g_mv += cv[k];
                g_en += 0.5 * (cu[k] * cu[k] + cv[k] * cv[k]);
            }

            #pragma omp single
            printf("%8.4f %12.6e %12.6e %14.8e %14.8e %14.8e\n",
                   n * dt, g_um, g_vm, g_mu * dA, g_mv * dA, g_en * dA);
        }
```

```
        if (n == nsteps) break;

        double t0 = 0.0;
        if (tid == 0) t0 = omp_get_wtime();

        #pragma omp for schedule(static)
        for (int i = 0; i < N; i++)
            calc_linha(cu, cv, nu_, nv_, N, i, c->r);

        if (tid == 0) c->t_passo += omp_get_wtime() - t0;

        const double *tu = cu; cu = nu_; nu_ = (double *)tu;
        const double *tv = cv; cv = nv_; nv_ = (double *)tv;
    }

    c->cpu_fim[tid] = ult;
    c->migr[tid]   = mig;
}

static const char *nome_politica(omp_proc_bind_t p)
{
    switch (p) {
        case omp_proc_bind_false: return "false";
        case omp_proc_bind_true: return "true";
        case omp_proc_bind_master: return "master";
        case omp_proc_bind_close: return "close";
        case omp_proc_bind_spread: return "spread";
        default:                return "?";
    }
}

static const char *env_ou(const char *nome)
{
    const char *s = getenv(nome);
    return s ? s : "(nao definido)";
}

int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <t_final> [N] [env|master|close|spread]\n", argv[0]);
        return 1;
    }
    const double tf = atof(argv[1]);
    if (tf <= 0.0) { fprintf(stderr, "t_final deve ser > 0\n"); return 1; }

    const int N = (argc >= 3) ? atoi(argv[2]) : N_GRID;
    if (N < 3) { fprintf(stderr, "N deve ser >= 3\n"); return 1; }

    const char *bind_arg = (argc >= 4) ? argv[3] : "env";
    Bind bind;
    if      (!strcmp(bind_arg, "env")) bind = B_ENV;
    else if (!strcmp(bind_arg, "master")) bind = B_MASTER;
    else if (!strcmp(bind_arg, "close")) bind = B_CLOSE;
    else if (!strcmp(bind_arg, "spread")) bind = B_SPREAD;
    else {
        fprintf(stderr, "bind invalido '%s'. Use env|master|close|spread "
                        "(false/true: OMP_PROC_BIND no ambiente)\n", bind_arg);
        return 1;
```

```
    }

    const double nu = NU;
    const int threads = omp_get_max_threads();

    const double L = 1.0;
    const double dx = L / N;
    const double r = 0.2;
    double dt     = r * dx * dx / nu;
    int   nsteps  = (int)ceil(tf / dt);
    dt = tf / nsteps;
    const double r_real = nu * dt / (dx * dx);

    const char *bind_efetivo = (bind == B_ENV) ? nome_politica(omp_get_proc_bind())
                                               : bind_arg;

    printf("Modo %d | N=%d | nu=%g | dx=%g | dt=%g | passos=%d | r=%.4f (limite 0.25)\n
        ",
           MODO, N, nu, dx, dt, nsteps, r_real);
    printf("OpenMP: threads=%d | schedule=static | versao=tarefa13 (regiao unica)\n",
        threads);
    printf("AFINIDADE: OMP_PROC_BIND=%s | OMP_PLACES=%s | clausula=%s | efetivo=%s |
        lugares=%d\n",
           env_ou("OMP_PROC_BIND"), env_ou("OMP_PLACES"), bind_arg,
           bind_efetivo, omp_get_num_places());
    if (r_real > 0.25) { fprintf(stderr, "Instavel! r > 0.25\n"); return 1; }

    size_t bytes = (size_t)N * N * sizeof(double);
    double *u = malloc(bytes), *v = malloc(bytes);
    double *un = malloc(bytes), *vn = malloc(bytes);
    int   *cpu_ini = malloc(threads * sizeof(int));
    int   *cpu_fim = malloc(threads * sizeof(int));
    long  *migr    = calloc(threads, sizeof(long));
    if (!u || !v || !un || !vn || !cpu_ini || !cpu_fim || !migr) {
        fprintf(stderr, "Sem memoria\n"); return 1;
    }
    for (int t = 0; t < threads; t++) { cpu_ini[t] = -1; cpu_fim[t] = -1; }

    const double A = 1.0, s0 = 0.05;
    const double xc = 0.5 * L, yc = 0.5 * L;

    /* Inicializacao paralela */
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double x = (i + 0.5) * dx, y = (j + 0.5) * dx;
            double r2 = (x - xc) * (x - xc) + (y - yc) * (y - yc);
            u[IDX(i, j, N)] = A * exp(-r2 / (2 * s0 * s0));
            v[IDX(i, j, N)] = 0.0;
            un[IDX(i, j, N)] = 0.0;
            vn[IDX(i, j, N)] = 0.0;
        }
    }

    printf("%8s %12s %12s %14s %14s %14s\n",
           "t", "max|u|", "max|v|", "mom_u", "mom_v", "energia");

    Ctx ctx = {
```

```
        .u = u, .v = v, .un = un, .vn = vn,
        .N = N, .nsteps = nsteps,
        .saida = (nsteps / 5 < 1) ? 1 : nsteps / 5,
        .dt = dt, .r = r_real, .dA = dx * dx,
        .total = (long)N * N, .t_passo = 0.0,
        .cpu_ini = cpu_ini, .cpu_fim = cpu_fim, .migr = migr
    };

    double t_total0 = omp_get_wtime();
    pascal_start(1);

    switch (bind) {
        case B_MASTER: {
            #pragma omp parallel proc_bind(master)
            integra(&ctx);
            break;
        }
        case B_CLOSE: {
            #pragma omp parallel proc_bind(close)
            integra(&ctx);
            break;
        }
        case B_SPREAD: {
            #pragma omp parallel proc_bind(spread)
            integra(&ctx);
            break;
        }
        default: {
            #pragma omp parallel
            integra(&ctx);
            break;
        }
    }

    pascal_stop(1);
    double t_total = omp_get_wtime() - t_total0;

    const double *uf = (nsteps % 2) ? un : u;

    /* Validacao contra a solucao analitica */
    double s2 = s0 * s0 + 2 * nu * tf;
    double err_max = 0, err_l2 = 0, ref_l2 = 0;
    #pragma omp parallel for schedule(static) \
            reduction(max:err_max) reduction(+:err_l2, ref_l2)
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double x = (i + 0.5) * dx, y = (j + 0.5) * dx;
            double r2 = (x - xc) * (x - xc) + (y - yc) * (y - yc);
            double ex = A * (s0 * s0 / s2) * exp(-r2 / (2 * s2));
            double d = uf[IDX(i, j, N)] - ex;
            if (fabs(d) > err_max) err_max = fabs(d);
            err_l2 += d * d;
            ref_l2 += ex * ex;
        }
    }
    printf("\nComparacao com solucao analitica em t=%g:\n", tf);
    printf(" erro maximo    = %.3e\n", err_max);
    printf(" erro L2 relativo = %.3e\n", sqrt(err_l2 / ref_l2));
```

```c

    long migr_total = 0;
    printf("\nMAPA thread:cpu_inicio->cpu_fim (migracoes)\n");
    for (int t = 0; t < threads; t++) {
        migr_total += migr[t];
        printf(" %d:%d->%d (%ld)%s", t, cpu_ini[t], cpu_fim[t], migr[t],
               (t % 4 == 3 || t == threads - 1) ? "\n" : "");
    }

    printf("\nTempo nos passos de calculo: %.4f s | tempo total do laco: %.4f s\n",
           ctx.t_passo, t_total);
    printf("RESULT,%d,%d,%s,%d,%.6f,%.6f,%.3e,%ld\n",
           N, threads, bind_efetivo, nsteps, ctx.t_passo, t_total, err_max, migr_total)
               ;

    free(u); free(v); free(un); free(vn);
    free(cpu_ini); free(cpu_fim); free(migr);
    return 0;
}
```

Código 1: Código `tarefa13.c`: versão 4 com afinidade de threads.

```
#!/bin/bash
#SBATCH --job-name=tarefa13
#SBATCH --time=0-4:0

#SBATCH --partition=amd-512
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=256
#SBATCH --exclusive

#SBATCH --mail-user=israel.soares.118@ufrn.edu.br
#SBATCH --mail-type=ALL

# Tarefa 13: escalabilidade do Navier-Stokes (v4) com diferentes afinidades de threads.
# Use o MESMO no (particao) da tarefa 12: ajuste --partition acima se necessario.

SRC=tarefa13.c
EXE=./tarefa13
N=2048              # malha N x N
TF=0.001            # tempo final simulado (define o numero de passos)
REPS=5              # repeticoes por configuracao (a variacao entre repeticoes chegou a ~30%)
TIMEOUT=180         # limite por execucao (s); casos patologicos (ex.: master) sao abortados
BIND_ARG=env        # env = usa OMP_PROC_BIND do ambiente | master|close|spread = clausula proc_bind

RESULTADOS=resultados_t13.csv
RESUMO=resumo_t13.csv
LOGDIR=logs_t13

# Configuracoes: rotulo|OMP_PROC_BIND|OMP_PLACES (places vazio = padrao do runtime)
CONFIGS=(
    "sem_afinidade|false|"
    "true_cores|true|cores"
```

```
    "close_threads|close|threads"
    "close_cores|close|cores"
    "close_sockets|close|sockets"
    "spread_threads|spread|threads"
    "spread_cores|spread|cores"
    "spread_sockets|spread|sockets"
    "master_cores|master|cores"
    # Exemplo de lugares explicitos (ajustar a topologia do no, veja lscpu):
    # "spread_manual|spread|{0:64:1},{64:64:1}"
)

# --- Compila se o executavel nao existir ---
if [[ ! -x "$EXE" ]]; then
    echo "Compilando $SRC ..."
    gcc -O3 -Wall -fopenmp -o "${EXE#./}" "$SRC" -lm -lmpascalops || { echo "Falha na
        compilacao"; exit 1; }
fi

# --- Diagnostico da topologia do no ---
echo "=== No: $(hostname) ==="
lscpu | grep -E 'Model name|Socket|Core|Thread|NUMA'
echo

# --- CPUs realmente visiveis para cada execucao ---
# ATENCAO: com --ntasks=1 o Slurm prende o proprio script (e o nproc dele) a 1 CPU.
# Por isso cada execucao roda via srun com todas as CPUs do no e sem cpu-bind;
# o numero de CPUs e medido dentro de um passo do srun, nao no shell do job.
NCPUS_ALLOC=${SLURM_CPUS_ON_NODE:-$(nproc --all)}
SRUN_OPTS=(--ntasks=1 --nodes=1 --cpus-per-task="$NCPUS_ALLOC" --cpu-bind=none --mem
    =0)
NCPUS=$(srun "${SRUN_OPTS[@]}" nproc)
echo "CPUs alocadas: $NCPUS_ALLOC | CPUs visiveis no passo srun: $NCPUS"
if (( NCPUS < 2 )); then
    echo "ERRO: apenas $NCPUS CPU visivel; o estudo de escalabilidade nao faz sentido.
        Abortando."
    exit 1
fi

# --- Numeros de threads: potencias de 2 ate o total de CPUs ---
THREADS=()
for ((t = 1; t < NCPUS; t *= 2)); do THREADS+=("$t"); done
THREADS+=("$NCPUS")
echo "Threads: ${THREADS[*]}"

# Limpa variaveis que poderiam interferir
unset OMP_PROC_BIND OMP_PLACES GOMP_CPU_AFFINITY OMP_NUM_THREADS

mkdir -p "$LOGDIR"
echo "caso,rep,N,threads,proc_bind,passos,tempo_calculo_s,tempo_total_s,erro_max,
    migracoes" > "$RESULTADOS"

roda() {
    local rotulo=$1 bind=$2 places=$3 t=$4 rep=$5
    local log="$LOGDIR/${rotulo}_T${t}_r${rep}.log"
    local amb=("OMP_NUM_THREADS=$t" "OMP_PROC_BIND=$bind")
    [[ -n "$places" ]] && amb+=("OMP_PLACES=$places")

    timeout "$TIMEOUT" srun "${SRUN_OPTS[@]}" \
```

```
        env -u OMP_PLACES -u OMP_PROC_BIND "${amb[@]}" \
        "$EXE" "$TF" "$N" "$BIND_ARG" > "$log" 2>&1
    local status=$?

    if grep -q '^RESULT' "$log"; then
        grep '^RESULT' "$log" | sed "s/^RESULT,/$rotulo,$rep,/" >> "$RESULTADOS"
    else
        echo " [FALHA status=$status] $rotulo T=$t rep=$rep (veja $log)"
    fi
}

for cfg in "${CONFIGS[@]}"; do
    IFS='|' read -r rotulo bind places <<< "$cfg"
    for t in "${THREADS[@]}"; do
        echo "=== $rotulo | OMP_PROC_BIND=$bind OMP_PLACES=${places:-padrao} | threads=
            $t ==="
        for ((rep = 1; rep <= REPS; rep++)); do
            roda "$rotulo" "$bind" "$places" "$t" "$rep"
        done
    done
done

# --- Resumo: medias, speedup e eficiencia (base: sem_afinidade com 1 thread) ---
awk -F, 'NR == 1 { next }
{
    k = $1 "," $4
    sc[k] += $7; st[k] += $8; sm[k] += $10; cnt[k]++
    if (!(k in ord)) { ord[k] = ++n; chave[n] = k }
}
END {
    b = "sem_afinidade,1"
    base = (cnt[b] > 0) ? sc[b] / cnt[b] : 0
    print "caso,threads,tempo_calculo_medio_s,tempo_total_medio_s,speedup,eficiencia,
        migracoes_medias"
    for (i = 1; i <= n; i++) {
        k = chave[i]; split(k, a, ",")
        tm = sc[k] / cnt[k]
        sp = (base > 0) ? base / tm : 0
        printf "%s,%s,%.4f,%.4f,%.3f,%.3f,%.1f\n", a[1], a[2], tm, st[k] / cnt[k], sp,
            sp / a[2], sm[k] / cnt[k]
    }
}' "$RESULTADOS" > "$RESUMO"

echo
echo "Concluido. Dados brutos: $RESULTADOS | resumo: $RESUMO | logs: $LOGDIR/"
column -s, -t "$RESUMO" 2>/dev/null || cat "$RESUMO"
```

Código 2: Script `tarefa13.sh`: execução no NPAD.